



Applied Software Project Report

By

Reshma Satynarayan Neelam

**A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment
of the requirements for the degree of Master of Science in Computer Science**

September, 2026



Scaler Mentee Email ID : roshninilam@gmail.com

Thesis Supervisor : Naman Bhalla

Date of Submission : 16/09/2026

© The project report of Reshma Satynarayan Neelam is approved, and it is acceptable in quality and form for publication electronically

Certification

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

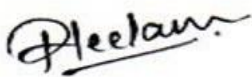
.....

Project Guide / Supervisor

DECLARATION

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from 23 February 2026 to 17 September 2026, is the result of my own individual endeavor. The Project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Reshma Satyanarayan Neelam

A handwritten signature in black ink, appearing to read 'Reshma', with a horizontal line underneath.

Date: 17 September 2026

ACKNOWLEDGMENT

I would like to thank **my family** for their patience during the many late nights this project demanded, and **my instructors and mentors at Scaler** Neovarsity for the guidance that shaped the way I now think about designing software. A particular thanks goes to my project module Instructors, **Naman Bhalla and Anurag Khanna**, whose feedback repeatedly pushed me to justify my design decisions rather than simply making them. I am also grateful to **my peers**, whose questions during review sessions exposed gaps I would otherwise have missed. Whatever I have learnt about building resilient, service-oriented systems is a direct result of this support.

Table of Contents

List of Tables	8
List of Figures	9
Applied Software Project.....	10
Abstract	10
Project Description.....	11
Requirement Gathering	15
2.1 Functional Requirements	15
2.2 Non-Functional Requirements	16
2.3 Users and Use Cases	16
2.4 Feature Set	18
2.5 Stakeholder Analysis	19
2.6 Development Milestones and Sprint Plan	20
2.7 Week-by-Week Milestone Review	21
2.8 Scope Constraints and Future Scope	22
Class Diagrams	23
3.1 Product Catalog Service Class Design.....	23
3.2 User Management Service Class Design.....	25
3.3 Order Processor Service Class Design	26
3.4 Payment Processor Service Class Design.....	27
3.5 Notification Service and Service Discovery Design	28
3.6 Layered Design Across Services.....	29
Database Schema Design.....	30
4.1 Conventions.....	30
4.2 Schema Ownership.....	31
4.2.1 Complete Implemented Schema Summary	34
4.2.2 Entity Relationships and Enforcement	35

4.2.3 Indexing, Constraints and Migration Control	36
4.3 Order State and Transitions	37
Feature Development Process	38
5.1 The problem	38
5.2 The request flow	38
5.3 How the flow maps to MVC	39
5.4 The correctness mechanisms	39
5.5 Verification	40
5.6 Performance Note	41
5.7 Cross-Service API Contract Summary	41
5.8 End-to-End Purchase and Event Flow	42
5.9 Test Strategy and Requirement Traceability	43
Deployment Flow	44
6.1 Deployment Security and Operational Boundaries.....	46
6.2 Current, Demonstration and Production-Shaped Deployment	47
Technologies Used	48
Conclusion.....	51
References	52
Supplementary Design and Implementation Appendices	53
Appendix A — Source Class Inventory	54
A.1 Product Catalog Source Inventory	54
A.2 User Management Source Inventory.....	55
A.3 Order Processor Source Inventory	56
A.4 Payment Processor Source Inventory	57
A.5 Notification Service Source Inventory.....	58
Appendix B — Column-Level Data Dictionary	59
B.1 Product Catalog Schema — Core Catalogue.....	59
product.....	59
category.....	59
product_category.....	59
B.2 Product Catalog Schema — Inventory and Events	60
product_inventory.....	60
inventory_reservation	60

product_image	60
outbox_event.....	60
B.3 User Management Schema — Identity and Sessions	61
users	61
user_credential.....	61
user_session	61
B.4 User Management Schema — Addresses, Roles and Events	62
user_address.....	62
role.....	62
user_role.....	62
outbox_event.....	62
B.5 Order Processor Schema — Order Aggregate.....	63
orders.....	63
order_item	63
order_address	63
B.6 Order Processor Schema — History and Reliability	64
order_status_history	64
order_idempotency.....	64
inbox_event.....	64
outbox_event.....	64
B.7 Payment Processor Schema	65
payment.....	65
webhook_event	65
outbox_event.....	65
Appendix C — Implemented Endpoint Catalogue	66
C.1 Product Catalog and User Management Endpoints	66
C.2 Order and Payment Endpoints	66
Appendix D — Additional Verification Cases	67
D.1 Additional Test Evidence.....	67

List of Tables

Table No.	Title	Page No.
2.01	Functional requirements by service	16
2.02	Non-functional requirements	17
2.03	Primary actors and their goals	18
2.04	Feature set of the platform	19
2.05	Stakeholder needs and Solution approach	20
2.06	Sprint overview and implementation status	21
2.07	Planned milestones and Deliverables	22
2.08	Scope decisions used during implementation	23
3.01	Layer implementation across the business services	30
4.01	Product Catalog owned tables	32
4.02	Implemented schema ownership summary	35
4.03	Important entity relationships	36
4.04	Database mechanisms and their purpose	37
4.05	Order State Machine transitions	38
5.01	Main implemented API groups	42
5.02	Purchase flow and safety mechanism at each step	43
5.03	Requirement-to-test traceability	44
6.01	Deployment security boundary	47
6.02	Deployment stages	48
7.01	Technology baseline summary	49

List of Figures

Figure No.	Title	Page No.
1.01	Project development process	12
1.02	System context and service communication	14
2.01	Use case diagram for the customer purchase journey	17
3.01	Product Catalog class diagram (hexagonal ports and adapters)	23
3.02	User Management core class relationships	26
3.03	Order Processor core class relationships.	27
3.04	Payment Processor core class relationships.	28
3.05	Notification and discovery class relationships.	29
4.01	Entity relationship diagram across the four stateful service schemas	34
5.01	Inventory reservation request flow	40
6.01	AWS single-host deployment topology	45

Applied Software Project

Abstract

Online retail platforms have to do several complex things at the same time: show an accurate catalogue, never sell stock they do not have, secure financial transactions, and keep the customer informed, all while remaining available under load. This project is my attempt to build such a platform the way it is actually built in industry — as a set of independently deployable microservices rather than one large application.

The system is built as five Spring Boot business services (Product Catalog, User Management, Order Processor, Payment Processor, and Notification) plus a Eureka service-discovery registry, all living in a Maven monorepo and communicating through a mix of narrow synchronous HTTP calls and asynchronous Kafka events. MySQL is the system of record, with each stateful service owning its own logical database and no cross-service foreign keys. Payments are handled through Stripe in test mode, so the platform makes genuine API and webhook calls that represent real life transactions of actual currency.

The core engineering ideas I wanted to demonstrate are idempotency (so a retried request never charges or reserves twice), the transactional outbox pattern (so a service never loses an event after committing its data), and saga-style compensation (so a failed step can be safely reversed instead of relying on a distributed transaction). The Notification service consumes order and payment events, and the Eureka registry ties them together. The feature I describe in depth is the concurrency-safe inventory reservation mechanism in Product Catalog, which I verified cannot oversell even when two orders race for the last unit of stock.

The practical value of this work is that the same patterns used here — service ownership, event-driven decoupling, and idempotent processing — are the patterns that let real e-commerce, ticketing, and banking systems stay correct and available. This report explains the requirements, the design, feature in depth, and how the system is intended to be deployed on AWS.

Project Description

The objective of this project is to design and build a backend for an e-commerce platform that behaves correctly under the conditions that actually break naive shopping systems: concurrent orders for the same item, retried network requests, payment provider callbacks arriving out of order, and services being temporarily unavailable. Rather than build one monolith and hope it scales, I chose a microservices architecture so that each business capability could be owned, deployed, and reasoned about on its own.

The platform is broken into five business services:

- **Product Catalog Service** — manages products, categories, images, stock levels, and the reservation/release of inventory for orders.
- **User Management Service** — handles signup, login, JWT issuance, profiles, addresses, and roles.
- **Order Processor Service** — the source of truth for an order's amount, currency, ownership, and lifecycle state; it orchestrates the purchase saga.
- **Payment Processor Service** — talks to Stripe, records payment and refund attempts, and ingests verified webhooks.
- **Notification Service** — consumes order and payment events and delivers emails, without ever blocking a business transaction.

Around these sit a **Service Discovery** component — a single-node Netflix Eureka registry that lets the services find each other by name rather than by hard-coded host — and the infrastructure dependencies — MySQL, a single Kafka broker, and a reverse proxy — which are not themselves "microservices" but the plumbing the services rely on.

A guiding decision throughout was to keep the scope honest for a learning project. Instead of trying to build all five services superficially, I built one runnable vertical slice at a time and only moved forward when the current slice passed its tests. The four core services — Product Catalog, User Management, Order Processor, and Payment Processor — are fully implemented, tested, and registered with Eureka; the Notification service is implemented as a durable Kafka consumer; and the remaining work (Login via IDP, Ecommerce interactive UI) is future scope.

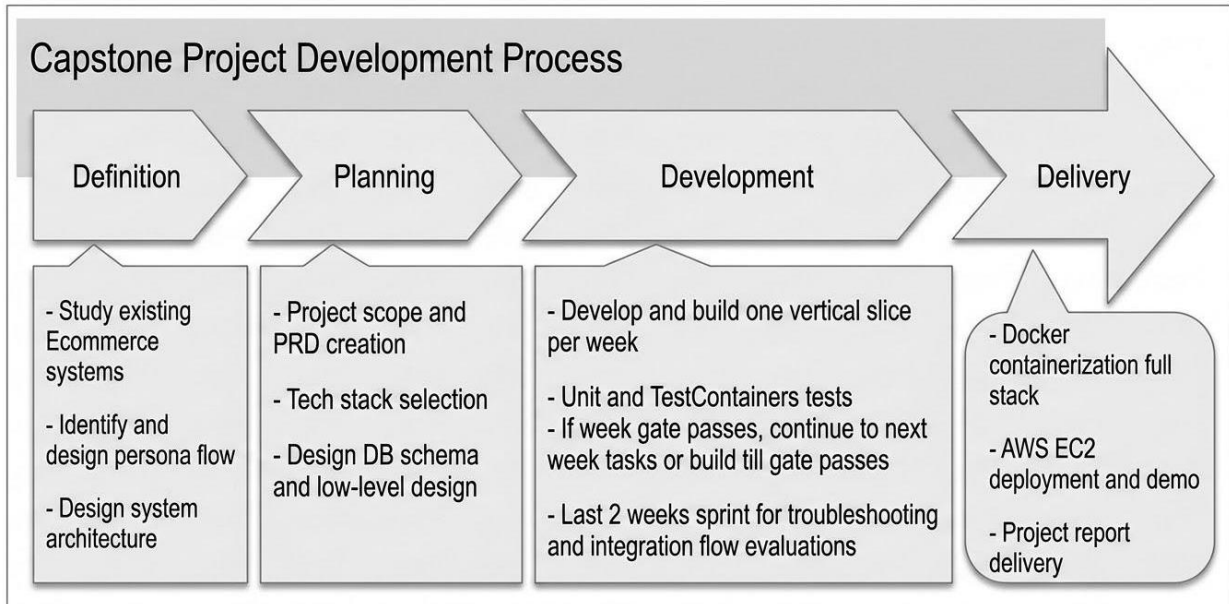


Figure 1.01: Project Development Process

The development of the **E-Commerce Microservices Platform** followed a structured and iterative approach to ensure systematic implementation, continuous testing, and reliable integration of the individual services. The development process was divided into multiple stages, beginning with requirements and low-level design and progressing through incremental feature development, testing, integration, and deployment.

1. Requirements and Low-Level Design

The initial stage focused on understanding the functional and non-functional requirements of the e-commerce platform and defining the overall technical design. The responsibilities of individual microservices, their interactions, and the major business workflows were identified. Low-level design was also performed to establish the structure of the services and their core components before implementation.

2. Database Schema Design

After defining the requirements and service structure, the database schemas were designed according to the data ownership and responsibilities of the individual services. This stage established the required entities, relationships, and persistence structure before feature

implementation. The project uses MySQL as the persistent data store, with each stateful service maintaining ownership of its respective data.

3. Vertical Slice Development

The implementation was performed incrementally by developing **one runnable vertical slice at a time**. Rather than implementing all services simultaneously, each slice was developed as a complete piece of functionality and integrated with the components required for its execution. This approach helped maintain a manageable development scope and allowed individual features to be validated before moving to subsequent functionality.

4. Testing and Validation

Each completed vertical slice underwent multiple levels of testing, including **unit testing, slice-level testing, and Testcontainers-based integration testing**. These tests were used to verify business logic, service behaviour, and interaction with infrastructure components. Testing was treated as an integral part of the development process rather than being deferred until the final integration stage.

5. Development Gate

A development gate was applied after each implementation and testing cycle. If the required tests or acceptance conditions were not satisfied, development returned to the current vertical slice for troubleshooting, correction, and re-testing. If the gate was successfully passed, development proceeded to the next service or vertical slice. This iterative feedback mechanism helped identify and resolve issues at an early stage.

6. Full-Stack Integration

After the required service slices had been individually implemented and validated, the services and their supporting infrastructure were integrated into a complete application stack using **Docker Compose**. This provided a consistent environment for running the microservices together with their infrastructure dependencies, including the required databases and messaging components.

7. Deployment and Demonstration

The final stage involved preparing the complete containerized application for deployment and demonstration on an **AWS EC2 instance**. This stage verified the ability to run the integrated system in a cloud environment and provided the final setup for project demonstration. The overall process therefore progressed from requirements and design through incremental implementation and validation to full-stack integration and cloud deployment.

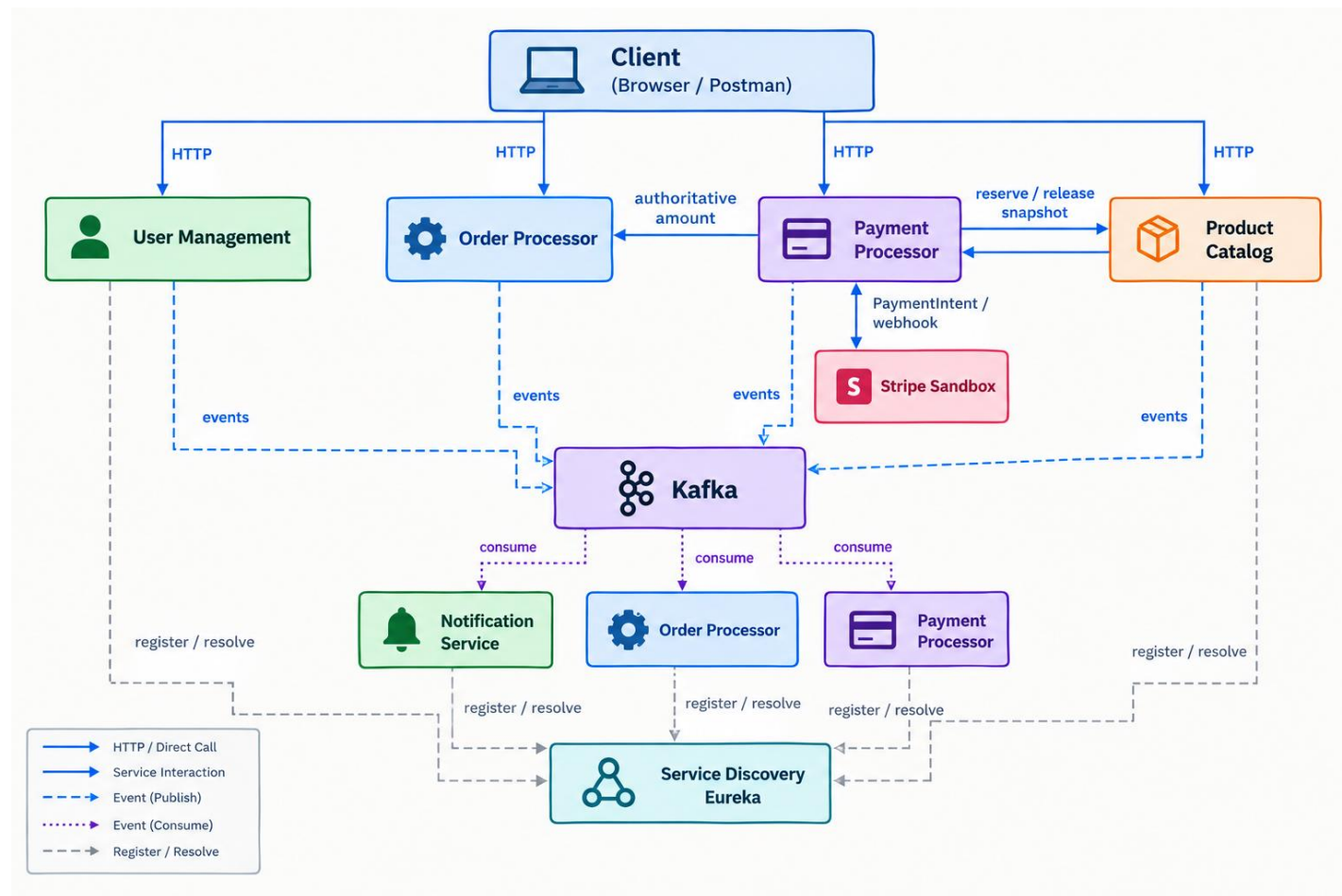


Figure 1.02: Microservices Architecture and Service Interaction Flow

This project has widespread relevance, almost every online business — retail, food delivery, ticketing, travel — solves exactly these problems. The specific techniques I implement here (idempotency keys, transactional outbox, optimistic and pessimistic locking, and saga compensation) are the same building blocks those companies use to avoid double-charging customers and overselling stock.

Requirement Gathering

2.1 Functional Requirements

The functional requirements were derived per service, since each service owns a distinct slice of the business.

Table 2.01: Functional requirements by service.

Service	Key functional requirements
Product Catalog	Create/update/soft-delete products and categories; upload image metadata; search products with bounded pagination and safe sorting; adjust on-hand stock; reserve and release inventory idempotently per order; expire stale reservations.
User Management	Signup with hashed passwords; login returning a short-lived JWT and a rotating refresh token; refresh and logout; retrieve/update profile; manage shipping and billing addresses; assign CUSTOMER/ADMIN roles.
Order Processor	Create an order from product IDs and quantities with an idempotency key; snapshot SKU, name, price, currency, and addresses; compute subtotal and total; expose order detail, history, and status; cancel orders subject to state rules.
Payment Processor	Create exactly one Stripe PaymentIntent per order using authoritative amounts loaded from Order; verify and deduplicate Stripe webhooks; record payment attempts; issue refunds on cancellation.
Notification	Consume order/payment events, deduplicate them, render a template, and deliver email with retry — without blocking any business transaction.

2.2 Non-Functional Requirements

Table 2.02: Non-functional requirements.

Category	Requirement
Correctness	Stock must never be oversold; a retried request must never have a second business effect.
Reliability	An event must not be lost after its business data is committed (transactional outbox); a consumer crash must result in safe redelivery, not data loss.
Security	Passwords stored only as bcrypt hashes; JWT access tokens short-lived; only Stripe test keys accepted in demo profiles; no raw card data ever reaches the backend.
Isolation	Each service owns its own database; no cross-service foreign keys or joins.
Observability	Every service exposes liveness/readiness probes; every mutating request carries a correlation ID.
Portability	The whole stack runs from one docker compose up on a laptop and on a single EC2 instance.
Maintainability	Code follows SOLID and a ports-and-adapters layout so providers (e.g. payment gateway) can be swapped without touching business logic.

2.3 Users and Use Cases

The system has three principal actors.

Table 2.03: Primary actors and their goals.

Actor	Goal
Customer	Browse products, place an order, pay for it, track its status, and cancel or get refunded when needed.
Administrator	Manage the catalogue and stock, and transition orders through fulfilment states.
External systems	Stripe (payment confirmation via webhook) and the email provider (delivery).

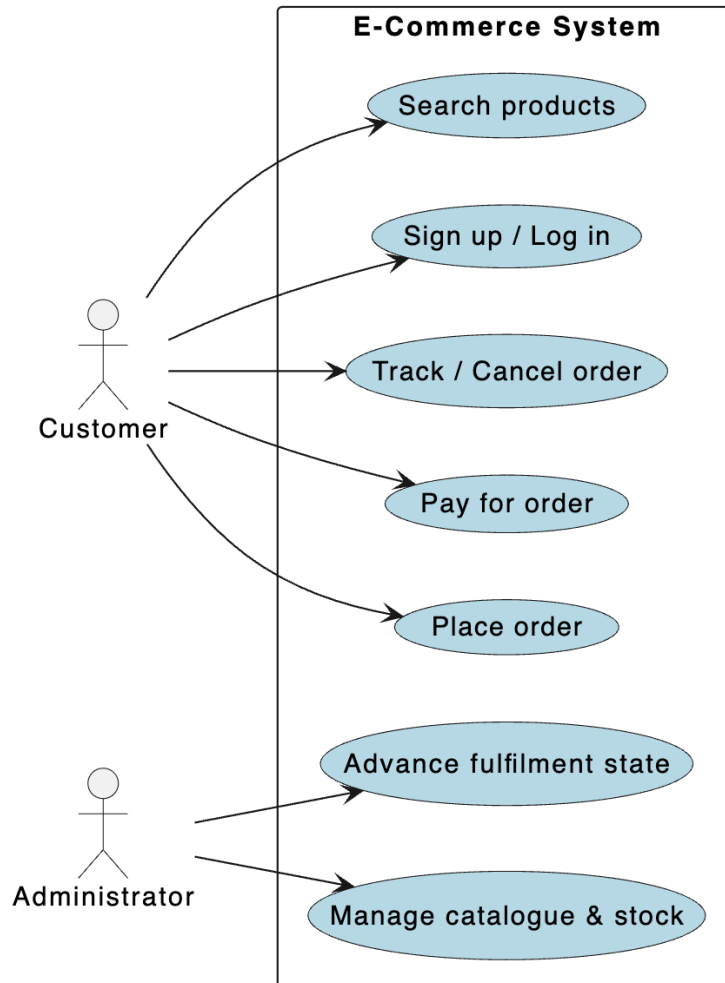


Figure 2.01: Use case diagram for the customer purchase journey.

A representative use case is *Place Order*: an authenticated customer submits product IDs and quantities with an idempotency key. The Order service fetches current product snapshots and asks Product Catalog to reserve stock. If reservation succeeds, the order is persisted as PENDING_PAYMENT and an OrderCreated event is written to the outbox in the same transaction. If the customer submits the same request again (a double-click or a network retry), the idempotency key ensures they get the original order back rather than a duplicate.

2.4 Feature Set

Table 2.04: Feature set of the platform.

Feature	Owning service
Product & category CRUD with soft delete	Product Catalog
Product search with bounded pagination and allow-listed sorting	Product Catalog
Concurrency-safe, idempotent inventory reservation & release	Product Catalog
Reservation expiry worker	Product Catalog
Transactional outbox for product/inventory events	Product Catalog
Signup / login / JWT / roles	User Management
Idempotent order creation with snapshots	Order Processor
Stripe test-mode payments with verified webhooks	Payment Processor
Kafka-driven order confirmation	Order + Payment
Cancellation and refund saga	Order + Payment
Durable, non-blocking notifications	Notification
Service discovery / registry	Service Discovery (Eureka)

2.5 Stakeholder Analysis

Table 2.05: Stakeholder requirements and solution approach

Stakeholder	Primary Requirements	Solution Approach
Customer	Secure account access, accurate products, safe ordering and payment status	JWT authentication, owner validation, inventory reservation, order snapshots and Stripe test-mode payment
Administrator	Controlled catalogue and stock maintenance	Role-aware product/category operations, stock adjustment endpoints and auditable status data
Application maintainer	Predictable builds, isolated services and diagnosable failures	Root Maven reactor, environment profiles, Actuator health, correlation IDs, outbox/inbox records and service-owned migrations
Payment provider	Correct amount, idempotent requests and verified callbacks	Payment reads the authoritative order amount, creates one logical payment and verifies Stripe webhook signatures
Notification provider	A clear message contract without blocking business work	Kafka consumer converts supported events to email/log messages after the order or payment transaction completes
Project guide / reviewer	Traceable claims and demonstrable implementation evidence	Requirements are mapped to source classes, Flyway tables and named automated tests

2.6 Development Milestones and Sprint Plan

Table 2.06: Sprint overview and implementation status.

Sprint	Weeks	Planned outcome	Implementation
Sprint 1: Foundation and Catalogue	1–2	Reproducible build, dependency stack and concurrency-safe catalogue	Parent POM, Maven wrapper, profiles, MySQL/Kafka Compose dependencies, full Product Catalog schema and reservation tests
Sprint 2: Identity and Orders	3–4	JWT identity plus idempotent order creation with snapshots	User Management and Order Processor modules, JWT/session/address flows, Product client, request hash, status history and ownership tests
Sprint 3: Payments and Messaging	5–6	Stripe test payment and Kafka-confirmed purchase	PaymentGateway/Stripe adapter, signed webhook processing, payment/order outboxes, Order inbox deduplication and event-flow integration tests
Sprint 4: Reversal, Notification and MVP	7–9	Cancellation/refund, durable notification and simple end-to-end demo	Reservation expiry and event- driven email/log sender exist; refund saga, durable notification jobs and user interface remain incomplete
Sprint 5: Packaging, Hosting and Operations	10–12	Complete Compose stack, EC2 demo, monitoring and recovery guidance	Health endpoints, profiles, correlation IDs and dependency Compose exist; service images, hosted deployment, central metrics and backup/restore proof remain future work

2.7 Week-by-Week Milestone Review

Table 2.07: Planned milestones and Deliverables.

Week	Milestone	Deliverable
1	Reproducible foundation	Six-module Maven reactor, Java 21, Spring Boot 3.5.16, H2 test profiles, Actuator and dependency Compose
2	Product Catalog	Seven active tables, CRUD/search, inventory reservation/release/expiry and concurrency coverage
3	User Management	Signup/login/refresh/logout, profile/address APIs, BCrypt, JWT and CUSTOMER/ADMIN roles
4	Order creation	Product snapshot client, stock reservation, monetary calculation, idempotency and status history
5	Stripe payment	Stripe port/adaptor, authoritative order lookup, PaymentIntent creation and verified webhook handling
6	Kafka purchase flow	Order/payment outbox relays, topics, PaymentEventConsumer, inbox deduplication and state-machine tests
7	Cancellation/refund	Order states and inventory expiry support recovery, complete refund orchestration
8	Notification	Kafka consumer, message factory and email/log senders; database-backed retry queue
9	Integrated demo	Bruno API collection; automated smoke flows
10	Containerized application	MySQL/Kafka dependencies are containerized; complete Compose deployment for application services
11	AWS deployment	Deployment design, hosted deployment artifacts or verification records
12	Hardening and recovery	Correlation IDs and broad automated tests; central metrics, outage runbook and restore proof

2.8 Scope Constraints and Future Scope

Table 2.08: Scope decisions used during implementation.

Constraint	Decision	Reason
Academic project duration	Build one vertical slice at a time	A tested purchase path provides stronger evidence than many incomplete modules
Local infrastructure	One MySQL server, one Kafka broker and one Eureka node	Sufficient to demonstrate service boundaries and messaging without claiming high availability
Payment access	Stripe test mode only	Exercises genuine intent/webhook behaviour without moving money
Performance features	No Redis cache in the current implementation	Correctness, schema ownership and retry safety were prioritised; caching is future scope
Deployment	Single-host EC2/Compose target	A realistic learning target that avoids premature orchestration complexity
Deferred features	Cart, Product reviews, Product promotions, shipment integration, UI and distributed tracing	Not required to prove the implemented backend patterns

Class Diagrams

The low-level design follows a hexagonal (ports-and-adapters) structure. The important idea is that application and domain code depend only on interfaces (ports); Spring infrastructure classes implement those interfaces (adapters). This is what makes the PaymentGateway swappable and keeps the CreateOrderService free of any WebClient or JpaRepository reference.

3.1 Product Catalog Service Class Design

The diagram below shows the Product Catalog service, used here as the representative example; the same ports-and-adapters structure is repeated across the other implemented services. A controller handles only HTTP concerns and delegates to a service; the service coordinates the business action and talks to repositories and the outbox writer through ports.

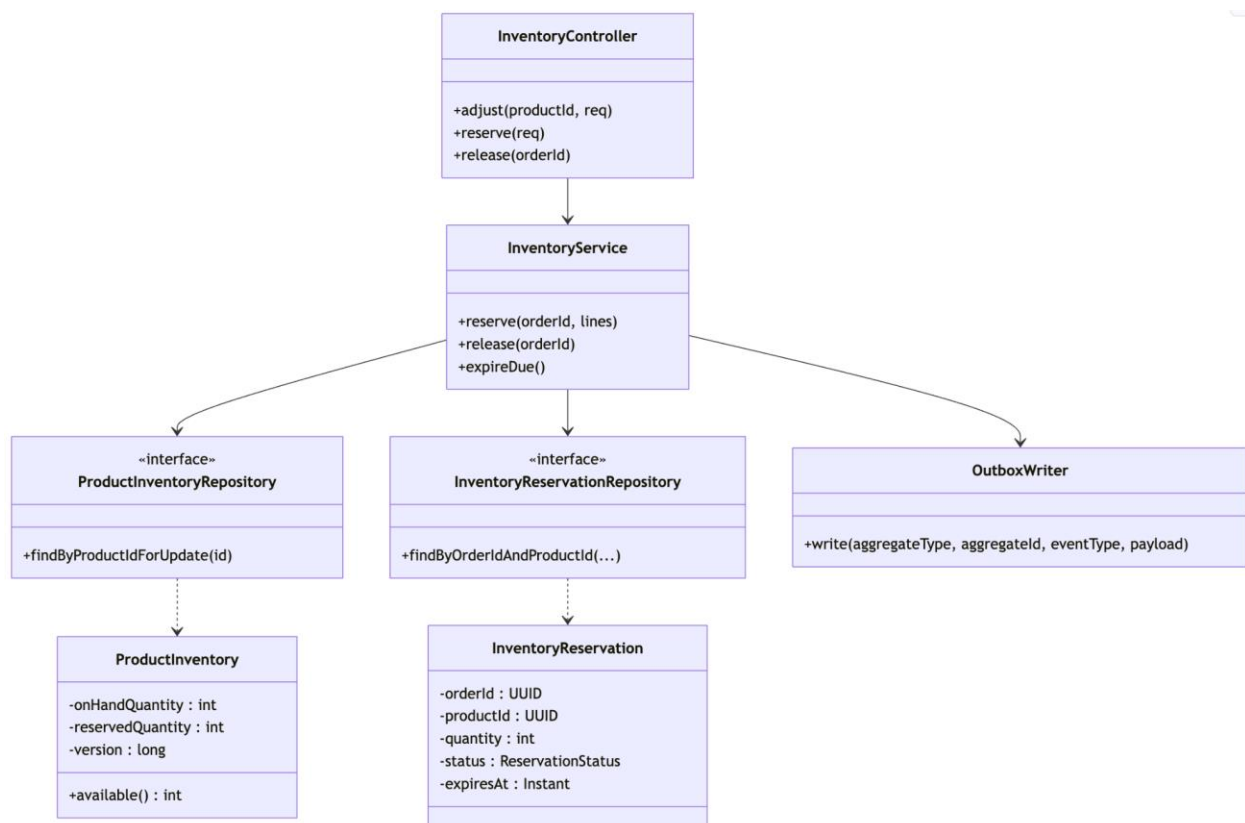


Figure 3.01: Product Catalog class diagram (hexagonal ports and adapters).

A few design rules are visible in this structure. The controller never contains business logic; overselling is prevented inside **InventoryService** using a pessimistic write lock combined

with an optimistic `@Version` column. The reservation record carries the `orderId` so that reserving twice for the same order is a no-op rather than a second deduction, and the `OutboxWriter` persists the domain event in the same transaction as the inventory change so the event can never be lost.

The same skeleton repeats across the other services. For example `Payment Processor` has a `PaymentController`, a `PaymentService`, and a `PaymentGateway` port whose only real implementation is `StripePaymentGateway`; in tests the port is replaced with a Mockito stub so the whole payment flow can be exercised without calling Stripe on every build. Keeping the port narrow is what lets me test the payment logic in isolation while still driving the genuine Stripe SDK from the integration profile.

3.2 User Management Service Class Design

User Management separates profile information, credentials and refresh sessions. AuthController handles signup and token operations, UserController handles the authenticated profile and addresses, and InternalUserController exposes only the small projection required by another service. The separation prevents password data from becoming part of normal profile queries.

Controllers	Services	Security / persistence	Domain
AuthController UserController InternalUserController	AuthenticationService UserProfileService AddressService	JwtAuthenticationFilter JwtTokenService TokenService 5 repositories	User UserCredential UserSession UserAddress Role

Relationship	Responsibility
AuthController → AuthenticationService	Signup, login, refresh rotation and logout.
AuthenticationService → PasswordEncoder / TokenService	BCrypt credential checks and JWT creation without exposing implementation in the controller.
UserController → UserProfileService / AddressService	Owner-scoped profile and address operations.
JwtAuthenticationFilter → JwtTokenService	Validates bearer tokens and creates AuthenticatedUser.

Figure 3.02: User Management core class relationships.

3.3 Order Processor Service Class Design

Order Processor is the orchestration point for the purchase. OrderService resolves live Product Catalog data, reserves inventory and checks the request idempotency key. OrderPersistenceService saves the order aggregate and the outbox event in one transaction. PaymentEventHandler later applies payment facts after InboxEvent has protected the consumer from duplicate Kafka delivery.

Controllers	Application	Clients / persistence	Domain
OrderController InternalOrderController InternalOutboxController	OrderService OrderPersistenceService OrderStateMachine PaymentEventHandler / Consumer	ProductClient 7 repositories OutboxWriter OutboxRelayPublisher	Order OrderItem OrderAddress OrderStatusHistory OrderIdempotency InboxEvent OutboxEvent

Relationship	Responsibility
OrderController → OrderService	Create, list and retrieve an order owned by the authenticated user.
OrderService → ProductClient	Fetch immutable product snapshots and reserve stock.
OrderService → OrderPersistenceService	Persist the aggregate, request hash and outbox event atomically.
PaymentEventConsumer → PaymentEventHandler	Separate Kafka acknowledgement from transactional state handling.
PaymentEventHandler → InboxEventRepository / OrderStateMachine	Deduplicate the event and validate the next order state.

Figure 3.03: Order Processor core class relationships.

3.4 Payment Processor Service Class Design

Payment Processor uses a port-and-adapter boundary around Stripe. `PaymentService` reads the owner, amount, currency and payable state from Order Processor; the client cannot choose the charge amount. `StripePaymentGateway` is the provider adapter, while `PaymentWebhookService` records the provider event ID before changing the local payment state and producing an outbox event.

Controllers	Application	Ports / persistence	Domain
PaymentController StripeWebhookController InternalOutboxController	PaymentService PaymentWebhookService PaymentStateMachine OutboxRelayPublisher	PaymentGateway StripePaymentGateway OrderPaymentDetailsClient 3 repositories	Payment WebhookEvent OutboxEvent CreatePaymentCommand CreatePaymentResult

Relationship	Responsibility
PaymentController → PaymentService	Create one logical payment and return an owned payment.
PaymentService → OrderPaymentDetailsClient	Obtain the authoritative amount and validate the order state.
PaymentService → PaymentGateway	Create the external intent without coupling the use case to the Stripe SDK.
StripeWebhookController → PaymentWebhookService	Apply only signature-verified provider facts.
PaymentWebhookService → WebhookEventRepository	Use provider_event_id as the webhook deduplication key.

Figure 3.04: Payment Processor core class relationships.

3.5 Notification Service and Service Discovery Design

Notification Service is deliberately smaller than the stateful services. It receives an EventEnvelope, maps supported order/payment events to NotificationMessage and sends through the configured NotificationSender implementation. The current module has no database-backed retry queue, so that is reported as a limitation rather than shown as implemented. Service Discovery contains the Eureka server entry point and keeps only a live registry, not business data.

Kafka input	Message mapping	Output port	Discovery
NotificationEventConsumer EventEnvelope	NotificationMessageFactory NotificationMessage	NotificationSender EmailNotificationSender LogNotificationSender	ServiceDiscovery Application Eureka registry

Figure 3.05: Notification and discovery class relationships.

3.6 Layered Design Across Services

The names vary by domain, but the dependency direction remains the same. A web or messaging adapter delegates to an application service; the service uses repository, HTTP-client or provider ports; and the domain model remains owned by one module. DTO classes remain at the boundary and are not used as JPA entities.

Table 3.01: Layer implementation across the business services.

Layer	Product	User	Order	Payment	Notification
Input adapter	REST controllers / scheduler	REST controllers	REST + Kafka consumer	REST + Stripe webhook	Kafka consumer
Application	Product, Category, Inventory services	Authentication, Profile, Address services	OrderService, persistence service, state machine	PaymentService, webhook service, state machine	Message factory
Output adapter	JPA repositories / outbox	JPA repositories / token service	Product client / JPA / Kafka relay	Order client / Stripe / JPA / Kafka relay	Email or log sender
Owned data	7 tables	7 tables	7 tables	3 tables	4 tables

Database Schema Design

The data model follows one strict boundary rule: **each stateful microservice owns a separate MySQL schema, and physical foreign keys exist only within the owning service.** An identifier that points at another service (for example a `user_id` on an order) is stored as a plain `CHAR(36)` with no FK constraint, and it is validated at the API or event boundary rather than by the database. This is what actually enforces service independence — we cannot accidentally write a query that joins two services together.

4.1 Conventions

- **Identifiers:** application-generated `CHAR(36)` UUID strings, mapped in Java as `java.util.UUID`. Slightly wasteful on space but very easy to inspect by hand.
- **Time:** `DATETIME(6)` stored in UTC.
- **Money:** `DECIMAL(19,4)` plus a `CHAR(3)` ISO-4217 currency code — never `FLOAT` or `DOUBLE`.
- **Optimistic locking:** aggregate tables carry a `version BIGINT` used with JPA `@Version`.
- **Deletion:** products are soft-deleted so old order snapshots stay explainable; orders and payments are retained, never deleted.
- **Reliability:** producing services keep an `outbox_event` table; a consuming service that must not double-process keeps an `inbox_event` table (Order Processor) or a provider-event dedup table (Payment Processor's `webhook_event`), while the Notification consumer deduplicates by dropping any event it cannot process.

4.2 Schema Ownership

product_catalog_db owns: product, category, product_category, product_inventory, inventory_reservation, product_image, outbox_event.

user_management_db owns: users, user_credential, user_session, user_address, role, user_role, outbox_event.

order_processor_db owns: orders, order_item, order_address, order_status_history, order_idempotency, inbox_event, outbox_event.

payment_processor_db owns: payment, webhook_event, outbox_event. (Payment attempts and refunds are recorded on the payment row and its lifecycle state; the inbound Stripe webhook is deduplicated through webhook_event.)

The Notification service and the Service Discovery (Eureka) registry are **stateless** and own no schema: Notification consumes events straight from Kafka and renders/sends them, deduplicating by dropping any event it cannot process rather than persisting an inbox, and Eureka simply holds an in-memory registry of live instances.

As a worked textual example, the core Product Catalog tables are described below in the same style used in the schema design document.

Table 4.01: Product Catalog owned tables (selected columns).

product		
- id	CHAR(36)	Primary Key
- sku	VARCHAR(64)	Unique
- name	VARCHAR(255)	
- status	VARCHAR(32)	ACTIVE INACTIVE DRAFT DISCONTINUED
- base_price	DECIMAL(19,4)	
- currency	CHAR(3)	
- deleted_at	DATETIME(6)	soft-delete marker
- version	BIGINT	optimistic lock

```

product_inventory
- product_id          CHAR(36)   Primary Key, FK -> product.id (one
row per product)
- on_hand_quantity    INT
- reserved_quantity   INT          available = on_hand - reserved
- version              BIGINT
- CHECK(reserved_quantity <= on_hand_quantity)

```

```

inventory_reservation
- id                  CHAR(36)   Primary Key
- order_id            CHAR(36)   external reference to Order service (no
FK)
- product_id          CHAR(36)   FK -> product.id
- quantity            INT
- status              VARCHAR(32) ACTIVE | RELEASED | CONSUMED | EXPIRED
- expires_at          DATETIME(6)
- UNIQUE(order_id, product_id)    <- this is what makes
reservation idempotent

```

Foreign Keys (within Product Catalog only):

- product_category(product_id) refers product(id)
- product_category(category_id) refers category(id)
- product_inventory(product_id) refers product(id)
- inventory_reservation(product_id) refers product(id)

Cardinality of relations:

- product to category -> m:m (through product_category)
- product to inventory -> 1:1
- product to reservation -> 1:m

The `UNIQUE(order_id, product_id)` constraint on `inventory_reservation` is small but important: it is the database-level guarantee that a retried reservation for the same order and product cannot create a duplicate hold.

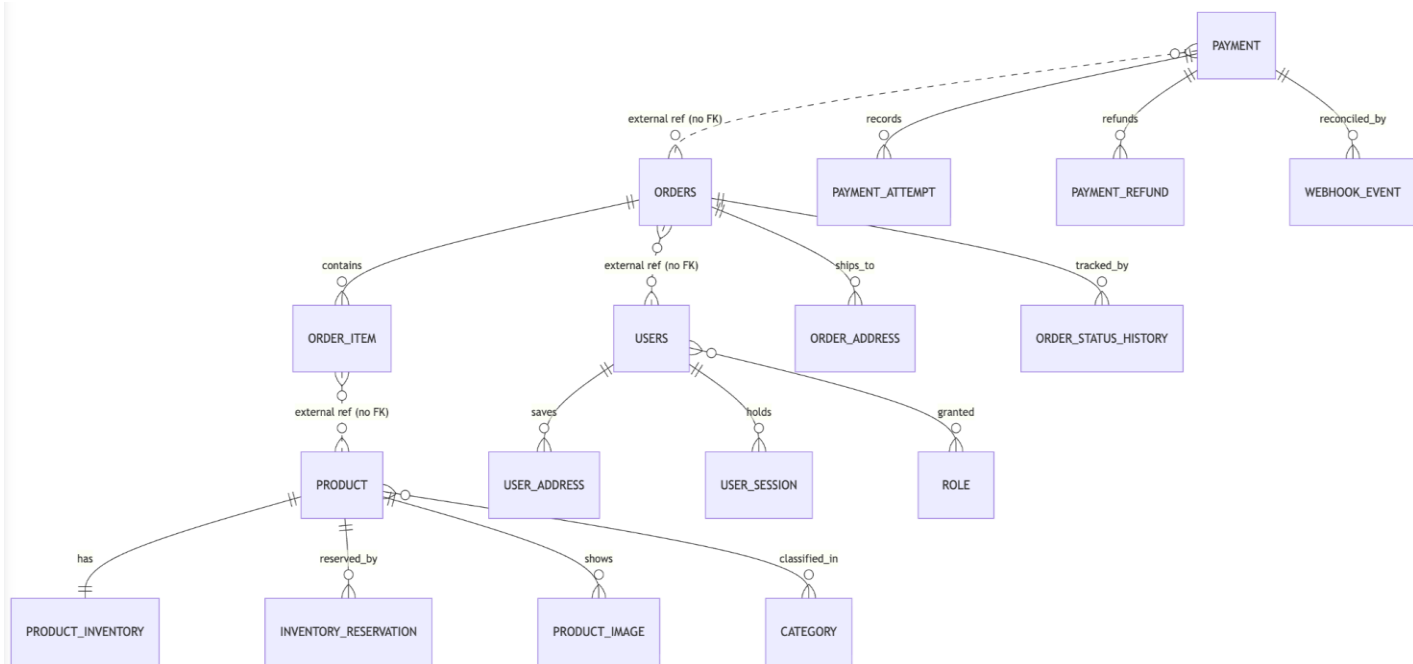


Figure 4.01: Entity relationship diagram across the four stateful service schemas.

Diagrammatically, the cross-service picture looks like the above figure. Solid lines are real foreign keys inside one schema; dashed lines are logical references across services that are *not* enforced by the database.

4.2.1 Complete Implemented Schema Summary

The active service-local Flyway migrations are the executable source of truth. Product Catalog, User Management and Order Processor each own seven tables, while Payment Processor owns three. Notification Service and Service Discovery are stateless in the current code. This distinction avoids presenting the future-looking reference SQL as if every table were already used by the application.

Table 4.02: Implemented schema ownership summary

Owner	Active tables	Logical cross-service references
Product Catalog	product, category, product_category, product_inventory, inventory_reservation, product_image, outbox_event	inventory_reservation.order_id → Order
User Management	users, user_credential, user_session, user_address, role, user_role, outbox_event	No required physical reference outside the schema
Order Processor	orders, order_item, order_address, order_status_history, order_idempotency, inbox_event, outbox_event	orders.user_id → User; order_item.product_id → Product; latest_payment_id → Payment
Payment Processor	payment, webhook_event, outbox_event	payment.order_id → Order; payment.user_id → User
Notification / Discovery	No active Flyway tables	Consumes events or maintains live registration only

4.2.2 Entity Relationships and Enforcement

A physical foreign key is used only when the parent and child are owned by the same service. UUIDs that identify another service's aggregate are logical references and are validated at an API or event boundary. This rule protects independent schema evolution and prevents accidental cross-service joins.

Table 4.03: Important entity relationships.

Relationship	Cardinality	Enforcement and reason
category parent → category	many-to-one	Self-referencing Product Catalog foreign key supports hierarchy
product ↔ category	many-to-many	product_category composite primary key and two local foreign keys
product → inventory	one-to-one	product_inventory.product_id is both primary and foreign key
product → reservation / image	one-to-many	Local foreign keys; reservation also has unique order/product key
user → credential	one-to-one	user_credential.user_id is both primary and foreign key
user → session / address	one-to-many	Local foreign keys retain ownership in User Management
user ↔ role	many-to-many	user_role composite primary key prevents duplicate assignment
order → item / address / history	one-to-many	Local foreign keys preserve the complete order aggregate
order ↔ payment	logical one-to-one	Unique payment.order_id; ownership and amount checked through Order API

4.2.3 Indexing, Constraints and Migration Control

To maintain data integrity and prevent race conditions across distributed microservices, the system relies on a combination of relational constraints, optimistic versioning, and pessimistic locking. These built-in database mechanisms work together to ensure that business operations remain safe, retries are fully idempotent, and schema migrations are applied reliably.

Table 4.04: Database mechanisms and their purpose.

Mechanism	Examples	Purpose
Primary and foreign keys	UUID aggregate keys; local child-table foreign keys	Prevent duplicate identity and orphaned rows within a service
Unique business keys	product.sku, users.email, order_number, idempotency key pairs, provider_event_id	Make business retries and external callbacks safe
Check constraints	quantity > 0; reserved_quantity between zero and on-hand	Reject invalid inventory and order data at the database boundary
Worker indexes	outbox status/created time; reservation status/expiry; inbox status/received time	Support bounded polling without scanning entire tables
Optimistic version	product, inventory, user, order and payment aggregates	Detect conflicting concurrent updates
Pessimistic lock	Product inventory SELECT FOR UPDATE	Serialise reservations for the same item and prevent overselling
Flyway versions	V1 baselines plus V2 customer_email additions	Apply schema changes in repeatable order without rewriting shared history

4.3 Order State and Transitions

The order lifecycle is driven by a pure state machine so that every transition is validated by domain code rather than scattered through controllers.

Table 4.05: Order state machine transitions.

From state	Allowed next states
PENDING_PAYMENT	CONFIRMED, PAYMENT_FAILED, CANCELLED
CONFIRMED	PROCESSING, CANCELLATION_PENDING
PROCESSING	SHIPPED, CANCELLATION_PENDING
SHIPPED	DELIVERED
DELIVERED	(terminal)
PAYMENT_FAILED	PENDING_PAYMENT, CANCELLED
CANCELLATION_PENDING	CANCELLED (refund failure keeps it pending for retry)
CANCELLED	(terminal)

Feature Development Process

The **inventory reservation** module was selected for detailed examination as it addresses a critical vulnerability in e-commerce architectures: preventing stock overselling under high concurrency. Furthermore, this component represents a fully realized implementation validated through rigorous testing, permitting an empirical description rather than a theoretical exposition.

5.1 The problem

When two customers try to buy the last unit of a product at nearly the same moment, both requests read "1 available", both decide there is enough stock, and both proceed — and now the system has sold two of something it has one of. The classic "read then write" pattern has a race condition. On top of that, network retries mean the *same* customer's reservation request might arrive twice, and the system must not treat that as two separate holds.

5.2 The request flow

The Order Processor calls the internal reservation endpoint on Product Catalog:

```
POST /internal/v1/inventory/reservations
{
  "orderId": "<order-uuid>",
  "lines": [ { "productId": "<uuid>", "quantity": 2 } ]
}
```

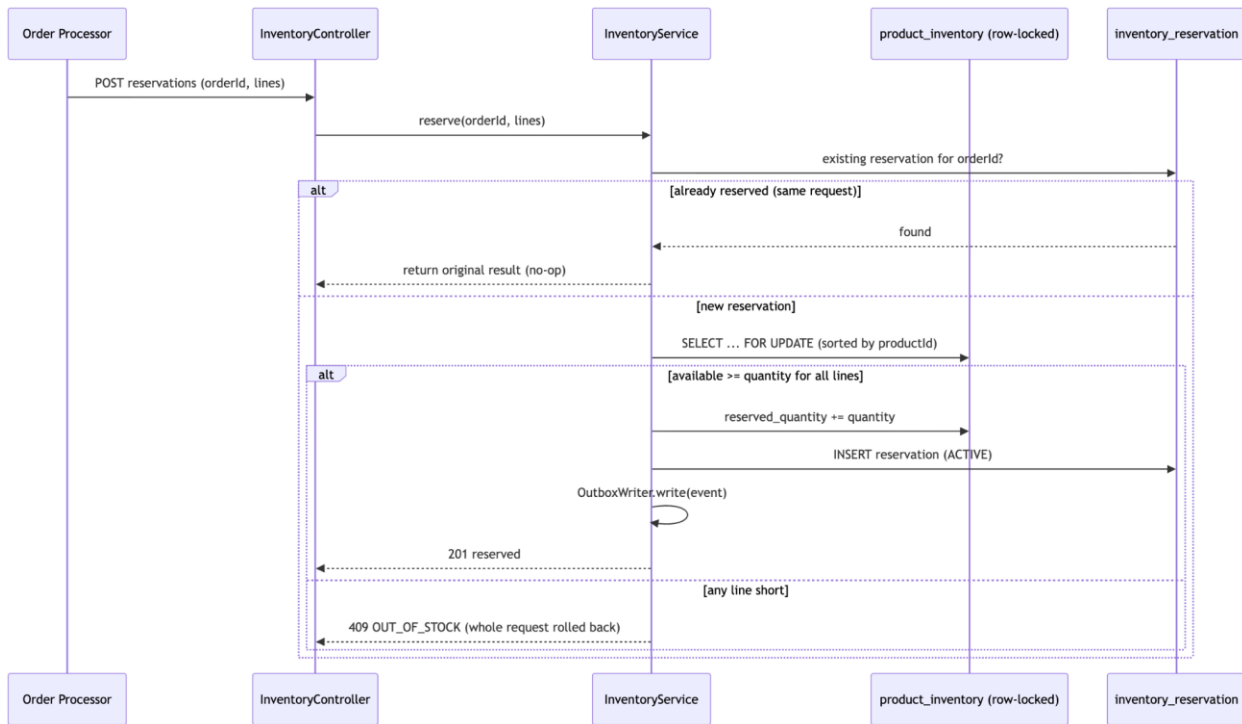


Figure 5.01: Inventory reservation request flow.

5.3 How the flow maps to MVC

1. **API request payload** — the JSON above, validated by bean validation (non-null order ID, positive quantities).
2. **Controller** — `InventoryController.reserve()` does only HTTP mapping and hands off to the service.
3. **Service** — `InventoryService.reserve()` holds the whole business decision inside one `@Transactional` method.

5.4 The correctness mechanisms

This project uses three mechanisms together, because no single one is sufficient:

- **Pessimistic row lock:** the service loads each inventory row with `SELECT ... FOR UPDATE (findByProductIdForUpdate)`. A concurrent reservation for the same

product blocks until the first transaction commits, so the second one reads the *already decremented* quantity.

- **Optimistic @Version:** a backstop on the aggregate that catches any lost-update that slips past the lock.
- **Idempotency on (order_id, product_id):** before doing anything, the service checks for an existing reservation for that order. A replayed request returns the original result. A replay with a *different* quantity for the same order returns 409 `DUPLICATE_REQUEST`, because that is a genuine conflict rather than a retry.

Multi-line reservations are all-or-nothing: the lines are sorted by product ID (to avoid deadlocks between two orders locking the same products in opposite order) and the entire transaction rolls back if any single line is short, so a customer never ends up with a partial hold.

5.5 Verification

The behaviour is proven by tests :

- `InventoryConcurrencyTest.concurrentReservations_cannotOverse`
`ll` fires two reservations at the last unit of stock and asserts exactly one succeeds and the other gets `OUT_OF_STOCK`.
- `InventoryReservationTest` covers idempotent replay (one business effect), release replay (no double increment), expiry running exactly once, and the different-quantity replay returning 409.
- `reserve_multiLine_isAllOrNothing` confirms partial reservations roll back.

The self-contained suite runs on in-memory H2 with no external dependency (`.\mvnw.cmd test` → 30 passed at the Week 2 gate), and a `Testcontainers` test applies the real Flyway migrations against a genuine MySQL 8.4 container during `mvn verify` when Docker is available. The same testing discipline extends across the monorepo: User Management, Order Processor, and Payment Processor each ship their own unit, slice, and `Testcontainers` migration tests (including `WireMock` tests for the cross-service HTTP clients and dedup tests for the inbox), so every service is proven the same way this one is.

5.6 Performance Note

The performance goal here is not raw throughput but **correctness under contention**. The pessimistic lock is scoped as tightly as possible — only the specific inventory rows are locked, only for the duration of the reservation transaction — so unrelated products are never blocked. Because reservation is idempotent, the Order service can safely retry on a timeout without inflating held stock, which removes an entire class of "phantom reserved quantity" bugs that would otherwise need a manual clean-up job. In other words, the optimisation scope here is the elimination of a correctness failure, measured by the concurrency test passing 100% of the time rather than by a millisecond figure.

5.7 Cross-Service API Contract Summary

The inventory feature sits inside a wider purchase contract. Public routes authenticate customers or administrators, while internal routes expose only the minimum information needed by another service. The client never supplies the final payment amount; Payment Processor reads it from Order Processor.

Table 5.01: Main implemented API groups.

Service	Endpoint group	Purpose
Product Catalog	GET/POST/PATCH/DELETE /api/v1/products and /categories	Search and administer catalogue data
Product Catalog	PUT /internal/v1/inventory/{productId}; POST/DELETE reservations	Adjust, reserve and release stock
User Management	POST /api/v1/auth/signup login refresh logout	Credential and token lifecycle
User Management	GET/PATCH /api/v1/users/me and address routes	Owner-scoped profile and addresses
Order Processor	POST/GET /api/v1/orders and status route	Idempotent creation and owned order retrieval
Order Processor	GET /internal/v1/orders/{id}/payment- details	Authoritative payment owner, amount, currency and state
Payment Processor	POST/GET /api/v1/payments	Create one payment and return owned payment state
Payment Processor	POST /webhooks/stripe	Receive and verify Stripe events
Order / Payment	POST /internal/v1/outbox/{id}/requeue	Recover a failed outbox record

5.8 End-to-End Purchase and Event Flow

Table 5.02: Purchase flow and safety mechanism at each step.

Step	Interaction	Safety mechanism
1	Customer → Order Processor	JWT identifies the owner; Idempotency-Key identifies a retry
2	Order → Product Catalog	Live product snapshot plus row-locked, all-or-nothing reservation
3	Order database transaction	Order aggregate, request hash and outbox event commit together
4	Order outbox → Kafka	Relay retries publication without rolling back the accepted order
5	Customer → Payment Processor	Payment reads authoritative order details and creates one provider intent
6	Stripe → webhook endpoint	Signature verification and unique provider event ID prevent invalid or duplicate effects
7	Payment outbox → Kafka	Payment result is published after local payment state commits
8	Order Kafka consumer	Inbox event ID and state machine protect against replay and illegal transitions
9	Notification consumer	Email/log delivery is outside the business transaction

The system implements an at-least-once processing architecture. While a request or event may be received multiple times, idempotency is enforced through database unique constraints, request hashes, inbox records, and webhook identifiers to ensure that each business operation is executed exactly once. Global exactly-once delivery guarantees are not claimed due to the independent failure domains of MySQL, Kafka, and Stripe.

5.9 Test Strategy and Requirement Traceability

This project uses a small test pyramid: pure unit tests for state and calculations, MVC tests for validation and authorization, WireMock for service clients, H2 integration tests for fast repository/use-case coverage, and separate MySQL/Kafka integration classes for behaviour that an in-memory database cannot prove.

Table 5.03: Requirement-to-test traceability.

Requirement	Tests	Verified behaviour
Stock is not oversold	InventoryConcurrencyTest	Two requests racing for the last unit cannot both succeed
Reservation replay is safe	InventoryReservationTest	Reserve/release/expiry have one business effect
Authentication lifecycle	AuthControllerTest; AuthFlowIntegrationTest	Signup, login, refresh, logout and invalid-token paths
Order creation is idempotent	OrderCreationIntegrationTest	Same key/body returns original; conflicting reuse is rejected
Order money is exact	OrderServiceMoneyTest	Decimal line totals and order totals
Order transitions are valid	OrderStateMachineTest	Only declared lifecycle changes are accepted
Payment uses Order authority	OrderPaymentDetailsClientWireMockTest	Remote contract and dependency failure mapping
Payment/webhook replay is safe	PaymentServiceTest; PaymentWebhookServiceTest	Idempotency key and provider event deduplication
Outbox failure is recoverable	OutboxRelayRecoveryIT; OutboxRelayPublisherTest	Failed publication remains visible and can be retried
Notification mapping is deterministic	NotificationMessageFactoryTest	Supported event payload becomes the expected message

Deployment Flow

The deployment target: the same stack is intended to run on a laptop and on a single AWS EC2 instance.

At the time of writing, the *dependency* stack — MySQL 8.4 and a single-node Kafka broker (KRaft, no ZooKeeper) — is containerised and brought up with one command (`docker compose -f compose.deps.yaml up -d`), while the five Spring services and the Eureka registry currently run as executable JARs built by Maven against that dependency stack. Packaging each service into its own image behind a single full-stack Compose file is the planned next step; the figure below shows that intended end state.

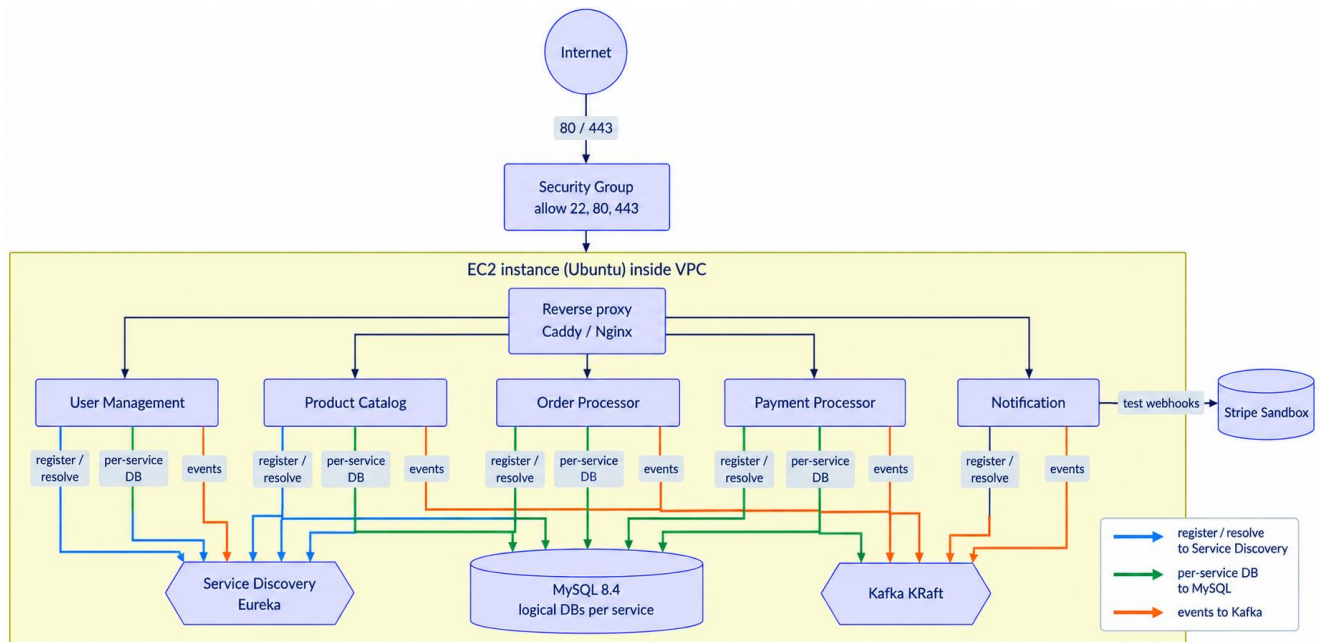


Figure 6.01: AWS single-host deployment topology.

EC2 (Elastic Compute Cloud). A single Ubuntu instance runs Docker Engine for the dependency stack and hosts the Spring services. In the intended end state all five services listen on container port 8080 and are reachable only through the private Compose network; only the reverse proxy publishes ports to the host. **Service discovery** is provided by the Netflix Eureka registry, so each

service registers itself on startup and resolves its peers by logical name rather than by a hard-coded host and port.

VPC (Virtual Private Cloud). The instance sits inside a VPC that gives it network isolation. The application services, MySQL, and Kafka never have their ports exposed to the internet — port 3306 and port 9092 stay on the private Compose network and are deliberately not published on the EC2 host.

Security Groups. The instance's security group is the firewall. It allows inbound 22 (SSH, ideally restricted to my own IP), 80, and 443 only. Everything else is denied, so the database and broker cannot be reached from outside.

RDS (Relational Database Service). For this learning stage MySQL runs as a pinned container with a named volume, which keeps the whole thing to one host and one `docker compose up`. In a production-shaped evolution this container would be replaced by a managed **RDS MySQL** instance, moving backups, patching, and failover to AWS while the application configuration changes only its `DB_HOST`.

Cache. The current design does not yet add a cache, but the natural place for one is the Product Catalog read path (product detail and search), where a managed **ElastiCache (Redis)** layer would cut database load for hot products. It is deliberately deferred until the purchase flow works, to avoid optimising something that is not yet a bottleneck.

Managed infrastructure. Beyond a single instance, the same containers could run on **Elastic Beanstalk** or a container service, which would add rolling deploys and health-based restarts. That is out of scope for this project, and building the per-service container images is the immediate next step — planned as multi-stage Dockerfiles with a non-root runtime user and explicit JVM memory bounds (`-XX:MaxRAMPercentage=70`).

Secrets (database passwords, the JWT signing key, and the Stripe test keys) are supplied through a `.env` file that is never committed. The demo profile refuses to start if it is given a live Stripe key, which is a small guard that makes it impossible to accidentally point the learning deployment at real money.

6.1 Deployment Security and Operational Boundaries

This section outlines the key security and operational controls implemented across the deployment architecture. It details the specific protective measures established for public ingress, administrative access, database isolation, and application security. These boundaries ensure that sensitive infrastructure, credentials, and business data remain protected while maintaining system traceability and health monitoring.

Table 6.01: Deployment security boundary.

Area	Intended control	Justification
Public ingress	Expose only HTTP/HTTPS through a reverse proxy in the hosted design	Keeps service ports out of the public attack surface
Administrative access	Restrict SSH to the developer IP	Avoids global port 22 exposure
Database and Kafka	Private container/network access; never publish ports on a public EC2 interface	Business data and event traffic are internal dependencies
Credentials	Environment variables and an ignored .env file; no committed secrets	Separates configuration from source control
Payment	Reject live Stripe keys in the learning profile and verify webhook signatures	Prevents accidental real-money use and forged callbacks
Application identity	JWT owner/role checks on customer APIs	Prevents one customer reading another customer's order or payment
Health	Actuator liveness/readiness endpoints	Supports startup checks and future container restart decisions
Traceability	Correlation ID filters across REST services	Links a request to logs and downstream work

6.2 Current, Demonstration and Production-Shaped Deployment

AWS security groups should permit only required inbound traffic, and an eventual RDS instance should be reachable only from the application security group. The database, broker and registry should not be exposed directly to the internet.

Table 6.02: Deployment stages

Stage	Components	Status / interpretation
Current local development	Six Spring modules, MySQL 8.4 and single-node Kafka dependency Compose, Eureka registry	Implemented for development and automated testing; services start through Maven/IDE profiles
Planned academic demonstration	One EC2 host, Docker Engine/Compose, private service/dependency network, reverse proxy, Stripe test webhook	Appropriate low-cost target
Production-shaped evolution	Managed RDS, replicated Kafka, multiple service instances, managed secrets, central metrics/traces	Future improvement requiring cost, operations and availability work beyond project scope

Technologies Used

Table 7.01: Technology baseline summary.

Area	Technology	Why it was chosen
Language / runtime	Java 21 (LTS)	Long-term support, records, and modern language features.
Framework	Spring Boot 3.5.x	Web, Validation, Security, Data JPA, and Actuator in one consistent stack.
Service discovery	Spring Cloud Netflix Eureka	Lets services register and resolve each other by logical name instead of hard-coded hosts.
Build	Maven multi-module monorepo	One parent POM pins versions; each service builds its own executable JAR.
Database	MySQL 8.4 (LTS) + Flyway	Reliable relational store; Flyway makes schema changes reproducible per service.
Messaging	Apache Kafka (KRaft, single node)	Durable, replayable event log for asynchronous decoupling.
Payments	Stripe (test mode)	Real API/webhook behaviour without moving real money or storing card data.
Testing	JUnit 5, Mockito, Testcontainers, WireMock	Fast unit tests plus real-container integration tests.
Packaging / deploy	Docker + Docker Compose on AWS EC2	One command brings up the whole stack locally and on the host.

Spring Boot. Spring Boot is the backbone of every service. It gives me dependency injection (which is what makes the ports-and-adapters design practical), declarative transactions with `@Transactional`, request validation, and production endpoints through Actuator. In real life the same framework runs a huge share of enterprise Java backends — banks, retailers, and streaming services — precisely because it standardises the boring parts and lets you focus on domain logic.

MySQL. MySQL is the system of record. Each service owns its own logical database, uses HikariCP for connection pooling, and manages its own schema through Flyway migrations. I rely on real relational features — unique constraints, check constraints, and row-level locking

(SELECT ... FOR UPDATE) — to enforce correctness that would be fragile if left to application code alone. MySQL powers a large part of the web, from content platforms to e-commerce catalogues.

Apache Kafka. Kafka is the asynchronous spine. Services publish domain events (OrderCreated, PaymentSucceeded, InventoryReservationExpired) to topics keyed by order ID, and other services consume them at their own pace. Because Kafka is a durable, replayable log, a consumer that was down simply catches up when it returns. This is exactly how large systems — ride-hailing, logistics tracking, fraud detection — move events between teams without coupling their databases together. In this project Kafka is what lets the Notification service exist without any business transaction ever waiting on it.

Spring Cloud Netflix Eureka. Eureka is the service registry. On startup each service registers itself with the registry, and the Order and Payment services resolve their peers (for example Product Catalog and Order) by logical service name rather than by a hard-coded address. This keeps inter-service HTTP calls from breaking every time a host or port changes, and it is the same discovery mechanism that underpins many Spring-based microservice estates in industry. It runs here as a single node, which is sufficient for this project scope.

Stripe (test mode). Stripe handles the actual payment mechanics. The backend only ever creates a PaymentIntent and reacts to a signed webhook; the card number is entered directly into Stripe's UI elements in the browser and never touches my server or database. Using test keys and test cards (4242 4242 4242 4242) means the full flow — intent, confirmation, webhook, refund — behaves like production without any real charge. This is the standard way modern startups add payments without taking on the full weight of PCI compliance themselves.

Docker and Docker Compose. Docker packages each service into an image built with a multi-stage Dockerfile, and Compose wires the services, MySQL, Kafka, and a reverse proxy into one network. The practical payoff is that "works on my machine" becomes "works anywhere Docker runs" — the identical Compose file starts the stack on my laptop and on EC2. Container orchestration in this style underpins a large fraction of cloud deployments today.

Testcontainers and WireMock. These make the tests trustworthy. Testcontainers spins up a real MySQL 8.4 (and later Kafka) container during mvn verify, so migrations and JPA mappings are validated against the real database rather than an approximation. WireMock stands in for outbound HTTP dependencies so I can assert things like outbound idempotency headers without calling the real provider on every build.

Conclusion

- **Key takeaways:** A primary insight gained through this project is that correctness in a distributed architecture requires intentional engineering rather than reliance on the database alone. Idempotency keys, the transactional outbox pattern, and saga-style compensation are not academic curiosities; they are the difference between a system that double-charges customers under retries and one that does not. Building the inventory reservation feature end to end, and then writing the concurrency test that actually tried to break it, taught me more about locking and transaction boundaries than any amount of reading had.
- **Practical applications:** Every technology used here maps directly to how real backends are built. Spring Boot and MySQL run a large share of enterprise systems; Kafka is the standard way large organisations move events between independent teams; Stripe is how a great many businesses accept payments without owning the risk of raw card data; and Docker is how nearly all of it gets shipped. The patterns — service ownership, event-driven decoupling, and idempotent processing — are exactly what keep e-commerce, ticketing, and banking platforms correct while remaining available.
- **Limitations and future improvements:** The deployment runs on a single host, so it is not highly available - if the EC2 instance dies, everything is down. The MySQL and Kafka nodes are single instances with no replication, and the Eureka registry is a single node. There is no cache layer, no distributed tracing backend, and no automated CI pipeline yet. On the cost side, a single small EC2 instance is cheap, but a production-shaped version — managed RDS with multi-AZ failover, ElastiCache, a multi-broker Kafka cluster, and an orchestrator — would cost meaningfully more and is deferred as this is sufficient for academic project scope. The clearest next steps are to move the database to managed RDS and add a Redis cache on the catalogue read path. Adding an interactive UI for actors to use the system.

References

1. Spring Boot Reference Documentation, Pivotal/VMware, <https://docs.spring.io/spring-boot/>
2. MySQL 8.4 Reference Manual, Oracle Corporation, <https://dev.mysql.com/doc/>
3. Testcontainers for Java Documentation, <https://java.testcontainers.org/>
4. Docker and Docker Compose Documentation, Docker Inc., <https://docs.docker.com/>
5. Flyway Documentation, Redgate, <https://documentation.red-gate.com/flyway>
6. JDK 21 Documentation, Oracle, <https://docs.oracle.com/en/java/javase/21/>
7. Spring Cloud Netflix Reference — Eureka Service Discovery, Broadcom, <https://docs.spring.io/spring-cloud-netflix/reference/>
8. Apache Kafka Design — Message Delivery Semantics, Apache Software Foundation, <https://kafka.apache.org/40/design/design/>
9. Stripe PaymentIntents API Reference, Stripe Inc., https://docs.stripe.com/api/payment_intents
10. Stripe Webhook Documentation, Stripe Inc., <https://docs.stripe.com/webhooks>
11. WireMock Stubbing and Request Matching Documentation, <https://wiremock.org/docs/stubbing/>
12. Amazon EC2 Security Groups, Amazon Web Services, <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html>
13. Amazon RDS for MySQL, Amazon Web Services, https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_MySQL.html

Books and Architecture References

1. Sam Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed., O'Reilly Media, 2021.
2. Martin Kleppmann, Designing Data-Intensive Applications, O'Reilly Media, 2017.
3. Chris Richardson, *Microservices Patterns*, Manning Publications, 2018 (transactional outbox, saga, and idempotent consumer patterns).

Supplementary Design and Implementation Appendices

Appendix	Coverage
A	Package-level production source-class inventory
B	Column-level data dictionary from active Flyway migrations
C	Implemented public and internal REST endpoint catalogue
D	Additional verification cases not available in the core chapters

Appendix A — Source Class Inventory

Chapter 3 presents the core class relationships. This appendix lists the remaining production classes by package so configuration, DTO, exception and support types can be located without making the main class diagrams unreadable.

A.1 Product Catalog Source Inventory

Package	Classes	Responsibility
application root	ProductCatalogApplication	Spring Boot entry point
config	SchedulingConfig	Security, HTTP client, Kafka and application configuration
controller	CategoryController, InventoryController, ProductController	REST adapters and endpoint mapping
dto	ApiErrorDTO, CategoryCreateRequestDTO, CategoryResponseDTO, CategoryUpdateRequestDTO, InventoryAdjustRequestDTO, InventoryResponseDTO, ReservationRequestDTO, ReservationResponseDTO, ImageCreateRequestDTO, ImageResponseDTO, ImageUpdateRequestDTO, ProductCreateRequestDTO, ProductResponseDTO, ProductUpdateRequestDTO	Validated request and response contracts
enums	CategoryStatus, OutboxStatus, ProductStatus, ReservationStatus	Supporting production classes
exception	ApiException, BadRequestException, ConflictException, GlobalExceptionHandler, NotFoundException	Domain/API errors and common response handling
mapper	CategoryMapper, InventoryMapper, ProductMapper	Entity-to-DTO conversion
model	Category, InventoryReservation, OutboxEvent, Product, ProductCategory, ProductCategoryId, ProductImage, ProductInventory	JPA entities and persisted state
observability	CorrelationId, CorrelationIdFilter	Correlation ID support
repository	CategoryRepository, InventoryReservationRepository, OutboxEventRepository, ProductCategoryRepository, ProductImageRepository, ProductInventoryRepository, ProductRepository	Spring Data persistence interfaces
service	CategoryService, CategoryServiceImpl, InventoryService, InventoryServiceImpl, OutboxWriter, ProductService, ProductServiceImpl, ReservationExpiryWorker	Use cases, transactions and domain coordination
specification	ProductSpecifications	Dynamic Product Catalog query predicates
web	PageableUtils	Web-layer validation helpers

A.2 User Management Source Inventory

Package	Classes	Responsibility
application root	UserManagementApplication	Spring Boot entry point
config	AdminBootstrapRunner, JwtProperties, PasswordEncoderConfig, SecurityConfig, SecuritySettingsProperties	Security, HTTP client, Kafka and application configuration
controller	AuthController, InternalUserController, UserController	REST adapters and endpoint mapping
dto	AddressRequestDTO, AddressResponseDTO, AddressUpdateRequestDTO, ApiErrorDTO, AuthTokensDTO, LoginRequestDTO, LogoutRequestDTO, RefreshRequestDTO, SignupRequestDTO, UserResponseDTO, InternalUserDTO, UpdateProfileRequestDTO	Validated request and response contracts
enums	AddressType, RoleName, UserStatus	Supporting production classes
exception	ApiException, BadRequestException, ConflictException, GlobalExceptionHandler, NotFoundException, UnauthorizedException	Domain/API errors and common response handling
model	Role, User, UserAddress, UserCredential, UserSession	JPA entities and persisted state
observability	CorrelationId, CorrelationIdFilter	Correlation ID support
repository	RoleRepository, UserAddressRepository, UserCredentialRepository, UserRepository, UserSessionRepository	Spring Data persistence interfaces
security	AuthenticatedUser, JwtAuthenticationFilter, JwtTokenService, RefreshTokenSupport, TokenService	JWT validation and authenticated-user representation
service	AddressService, AuthenticationService, UserProfileService	Use cases, transactions and domain coordination

A.3 Order Processor Source Inventory

Package	Classes	Responsibility
application root	OrderProcessorApplication	Spring Boot entry point
client	ProductClient, ProductSnapshotDTO, ReserveRequest	Synchronous clients for another service
config	JwtProperties, KafkaTopicsConfig, OrderPricingProperties, ProductCatalogProperties, RestClientConfig, SecurityConfig	Security, HTTP client, Kafka and application configuration
controller	InternalOrderController, InternalOutboxController, OrderController	REST adapters and endpoint mapping
dto	ApiErrorDTO, AddressDTO, OrderCreateRequestDTO, OrderItemResponseDTO, OrderResponseDTO, OrderStatusResponseDTO, PaymentDetailsResponseDTO	Validated request and response contracts
enums	AddressType, ChangedByType, InboxStatus, OrderStatus, OutboxStatus	Supporting production classes
exception	ApiException, BadRequestException, ConflictException, DependencyUnavailableException, GlobalExceptionHandler, NotFoundException, UnauthorizedException	Domain/API errors and common response handling
mapper	OrderMapper	Entity-to-DTO conversion
messaging	EventEnvelope, EventEnvelopeSerializer	Event envelope and serialization types
model	InboxEvent, Order, OrderAddress, OrderIdempotency, OrderItem, OrderStatusHistory, OutboxEvent	JPA entities and persisted state
observability	CorrelationId, CorrelationIdFilter	Correlation ID support
repository	InboxEventRepository, OrderAddressRepository, OrderIdempotencyRepository, OrderItemRepository, OrderRepository, OrderStatusHistoryRepository, OutboxEventRepository	Spring Data persistence interfaces
security	AuthenticatedUser, JwtAuthenticationFilter, JwtTokenService, TokenService	JWT validation and authenticated-user representation
service	OrderPersistenceService, OrderService, OrderStateMachine, OutboxRelayPublisher, OutboxWriter, PaymentEventConsumer, PaymentEventHandler, RequestHasher, ResolvedLine	Use cases, transactions and domain coordination

A.4 Payment Processor Source Inventory

Package	Classes	Responsibility
application root	PaymentProcessorApplication	Spring Boot entry point
client	OrderPaymentDetailsClient, OrderPaymentDetailsDTO	Synchronous clients for another service
config	JwtProperties, KafkaTopicsConfig, OrderClientProperties, RestClientConfig, SecurityConfig, StripeConfig, StripeProperties	Security, HTTP client, Kafka and application configuration
controller	InternalOutboxController, PaymentController, StripeWebhookController	REST adapters and endpoint mapping
dto	ApiErrorDTO, CreatePaymentRequestDTO, PaymentResponseDTO	Validated request and response contracts
enums	OutboxStatus, PaymentStatus	Supporting production classes
exception	ApiException, BadRequestException, ConflictException, DependencyUnavailableException, GlobalExceptionHandler, NotFoundException, UnauthorizedException	Domain/API errors and common response handling
gateway	CreatePaymentCommand, CreatePaymentResult, PaymentGateway, StripePaymentGateway	External payment-provider port and Stripe adapter
messaging	EventEnvelope, EventEnvelopeSerializer	Event envelope and serialization types
model	OutboxEvent, Payment, WebhookEvent	JPA entities and persisted state
observability	CorrelationId, CorrelationIdFilter	Correlation ID support
repository	OutboxEventRepository, PaymentRepository, WebhookEventRepository	Spring Data persistence interfaces
security	AuthenticatedUser, JwtAuthenticationFilter, JwtTokenService, TokenService	JWT validation and authenticated-user representation
service	OutboxRelayPublisher, OutboxWriter, PaymentService, PaymentStateMachine, PaymentWebhookService	Use cases, transactions and domain coordination

A.5 Notification Service Source Inventory

Package	Classes	Responsibility
application root	NotificationServiceApplication	Spring Boot entry point
consumer	NotificationEventConsumer	Kafka input adapter
messaging	EventEnvelope, NotificationMessage	Event envelope and serialization types
service	EmailNotificationSender, LogNotificationSender, NotificationMessageFactory, NotificationSender	Use cases, transactions and domain coordination

A.6 Service Discovery Source Inventory

Package	Classes	Responsibility
application root	ServiceDiscoveryApplication	Spring Boot entry point

Appendix B — Column-Level Data Dictionary

Chapter 4 explains schema ownership, relationships, constraints and migration control. The following data dictionary records individual columns from the active service-local Flyway migrations and is retained as reference material rather than repeated in the main discussion.

B.1 Product Catalog Schema — Core Catalogue

Product is the catalogue aggregate root. Category has an optional self-reference for hierarchy, while product_category resolves the many-to-many relationship. Product rows are soft deleted so an old order snapshot remains understandable even after a product is no longer sold.

product

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK, not null	Application-generated product UUID
sku	VARCHAR(64)	Unique, not null	Business-facing stock keeping unit
name	VARCHAR(255)	Not null, indexed	Product display name
description	TEXT	Nullable	Long description
brand	VARCHAR(128)	Nullable	Manufacturer or brand
status	VARCHAR(32)	Default ACTIVE	Lifecycle state
base_price	DECIMAL(19,4)	Not null	Authoritative selling price
currency	CHAR(3)	Not null	ISO-4217 currency
created_at / updated_at	DATETIME(6)	Not null	UTC audit timestamps
deleted_at	DATETIME(6)	Nullable	Soft-delete marker
version	BIGINT	Default 0	Optimistic locking version

category

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Category UUID
parent_category_id	CHAR(36)	FK → category.id	Optional parent in category tree
name	VARCHAR(128)	Not null	Category name
slug	VARCHAR(160)	Unique	URL-safe category identifier
status	VARCHAR(32)	Default ACTIVE	Lifecycle state
created_at / updated_at	DATETIME(6)	Not null	UTC audit timestamps

product_category

Column	Type	Key / constraint	Purpose
product_id	CHAR(36)	PK, FK → product.id	Joined product
category_id	CHAR(36)	PK, FK → category.id	Joined category
is_primary	BOOLEAN	Default false	Marks the main category
created_at	DATETIME(6)	Not null	Assignment time

B.2 Product Catalog Schema — Inventory and Events

Inventory is separated from product description because stock changes more frequently than catalogue text. A one-to-one product_inventory row holds counters. inventory_reservation records each order hold, and the unique order/product pair makes a repeated reservation idempotent.

product_inventory

Column	Type	Key / constraint	Purpose
product_id	CHAR(36)	PK, FK → product.id	One inventory row per product
on_hand_quantity	INT	Check ≥ 0	Physical stock recorded
reserved_quantity	INT	0 ≤ reserved ≤ on hand	Stock held for active orders
reorder_level	INT	Default 0	Low-stock threshold
updated_at	DATETIME(6)	Not null	Last stock change
version	BIGINT	Default 0	Optimistic locking version

inventory_reservation

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Reservation UUID
order_id	CHAR(36)	External reference	Order owner; deliberately no cross-schema FK
product_id	CHAR(36)	FK → product.id	Reserved product
quantity	INT	Not null	Held quantity
status	VARCHAR(32)	Default ACTIVE	ACTIVE, RELEASED, CONSUMED or EXPIRED
expires_at	DATETIME(6)	Indexed with status	Automatic release deadline
created_at / updated_at	DATETIME(6)	Not null	UTC audit timestamps
order_id + product_id	Composite	Unique	Database-level reservation idempotency

product_image

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Image metadata UUID
product_id	CHAR(36)	FK → product.id	Owning product
url	VARCHAR(1024)	Not null	External object URL
alt_text	VARCHAR(255)	Nullable	Accessible alternative text
display_order	INT	Default 0	Stable image ordering
is_primary	BOOLEAN	Default false	Primary image marker
created_at	DATETIME(6)	Not null	Creation time

outbox_event

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Event UUID
aggregate_type / aggregate_id	VARCHAR(64) / CHAR(36)	Indexed	Source aggregate identity
event_type	VARCHAR(128)	Indexed	Domain event name
payload_json	JSON	Not null	Serializable event body
status	VARCHAR(24)	Default NEW	Relay state
created_at / published_at	DATETIME(6)	Published nullable	Creation and delivery time
retry_count / last_error	INT / VARCHAR(1000)	Default 0 / nullable	Relay recovery details

B.3 User Management Schema — Identity and Sessions

Profile, credential and session data are separate because they have different access and lifecycle rules. The users row can be read for a profile without loading a password hash. A refresh token is stored only as a hash, which supports revocation without retaining the usable token value.

users

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	User UUID
email	VARCHAR(320)	Unique	Normalized login identity
first_name / last_name	VARCHAR(100)	Not null	Profile name
phone	VARCHAR(32)	Nullable	Contact number
status	VARCHAR(32)	Default ACTIVE	Account state
email_verified_at / last_login_at	DATETIME(6)	Nullable	Account activity dates
created_at / updated_at	DATETIME(6)	Not null	UTC audit timestamps
version	BIGINT	Default 0	Optimistic locking version

user_credential

Column	Type	Key / constraint	Purpose
user_id	CHAR(36)	PK, FK → users.id	One credential row per user
password_hash	VARCHAR(255)	Not null	BCrypt output; never stores plain text
password_algorithm	VARCHAR(32)	Default BCrypt	Hashing algorithm identifier
password_changed_at / updated_at	DATETIME(6)	Not null	Credential audit times

user_session

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Session UUID
user_id	CHAR(36)	FK → users.id	Session owner
refresh_token_hash	CHAR(64)	Unique	SHA-256 representation of refresh token
issued_at / expires_at	DATETIME(6)	Not null	Refresh token lifetime
revoked_at	DATETIME(6)	Nullable	Logout or rotation marker
client_fingerprint	VARCHAR(255)	Nullable	Optional client context

B.4 User Management Schema — Addresses, Roles and Events

A user may own several addresses and several roles. The user_role join table gives a conventional many-to-many mapping. CUSTOMER and ADMIN are inserted by the baseline migration so authorization has stable role identities in every environment.

user_address

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Address UUID
user_id	CHAR(36)	FK → users.id	Address owner
address_type	VARCHAR(24)	Default SHIPPING	Shipping or billing classification
recipient_name	VARCHAR(200)	Not null	Delivery recipient
line1 / line2	VARCHAR(255)	Line1 not null	Street address
city / state_region	VARCHAR(128)	City not null	Locality and region
postal_code / country_code	VARCHAR(32) / CHAR(2)	Not null	Postal and ISO country data
phone	VARCHAR(32)	Nullable	Delivery contact
is_default	TINYINT(1)	Default 0	Default-address marker
created_at / updated_at	DATETIME(6)	Not null	UTC audit timestamps

role

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Role UUID
name	VARCHAR(64)	Unique	CUSTOMER or ADMIN
description	VARCHAR(255)	Nullable	Human-readable purpose

user_role

Column	Type	Key / constraint	Purpose
user_id	CHAR(36)	PK, FK → users.id	Assigned user
role_id	CHAR(36)	PK, FK → role.id	Assigned role
assigned_at	DATETIME(6)	Not null	Assignment audit time

outbox_event

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Event UUID
aggregate_type / aggregate_id	VARCHAR(64) / CHAR(36)	Not null	Source user aggregate
event_type	VARCHAR(128)	Not null	Domain event name
payload_json	JSON	Not null	Event body
status	VARCHAR(24)	Default NEW	Relay state
created_at / published_at	DATETIME(6)	Published nullable	Creation and delivery time
retry_count / last_error	INT / VARCHAR(1000)	Recovery columns	Relay diagnostics

B.5 Order Processor Schema — Order Aggregate

The order database keeps snapshots instead of joining to Product Catalog or User Management. This is necessary because product names, prices and delivery addresses can change after purchase. Historical order presentation therefore remains stable and does not require another service to be online.

orders

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Order UUID
order_number	VARCHAR(32)	Unique	Customer-facing order number
user_id	CHAR(36)	External reference	Ordering user; no cross-schema FK
customer_email	VARCHAR(320)	Nullable, added by V2	Email snapshot for downstream events
status	VARCHAR(40)	Default PENDING_PAYMENT	Order state machine value
currency	CHAR(3)	Not null	Order currency
subtotal / discount / tax / shipping	DECIMAL(19,4)	Not null	Price components
total_amount	DECIMAL(19,4)	Not null	Final amount due
latest_payment_id	CHAR(36)	External reference	Latest Payment service identifier
created_at / updated_at / cancelled_at	DATETIME(6)	Cancelled nullable	Lifecycle timestamps
version	BIGINT	Default 0	Optimistic locking version

order_item

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Line UUID
order_id	CHAR(36)	FK → orders.id	Owning order
product_id	CHAR(36)	External reference	Original Product Catalog UUID
sku_snapshot / name_snapshot	VARCHAR(64) / VARCHAR(255)	Not null	Immutable product description at purchase
unit_price / line_total	DECIMAL(19,4)	Not null	Snapshot price and calculated line value
quantity	INT	Check > 0	Purchased quantity
currency	CHAR(3)	Not null	Line currency

order_address

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Address snapshot UUID
order_id	CHAR(36)	FK → orders.id	Owning order
address_type	VARCHAR(24)	Unique with order_id	One address per type per order
recipient_name	VARCHAR(200)	Not null	Recipient snapshot
line1 / line2	VARCHAR(255)	Line1 not null	Street snapshot
city / state_region	VARCHAR(128)	City not null	Locality snapshot
postal_code / country_code	VARCHAR(32) / CHAR(2)	Not null	Postal snapshot
phone	VARCHAR(32)	Nullable	Delivery contact snapshot

B.6 Order Processor Schema — History and Reliability

The remaining tables support traceability and safe retries. `order_status_history` is append-only audit data, `order_idempotency` gives a repeated create request the same result, `inbox_event` deduplicates payment events, and `outbox_event` publishes order changes after the database transaction commits.

`order_status_history`

Column	Type	Key / constraint	Purpose
id	BIGINT UNSIGNED	PK, auto increment	Ordered audit row
order_id	CHAR(36)	FK → orders.id	Tracked order
from_status / to_status	VARCHAR(40)	To status not null	State transition
reason_code / reason_text	VARCHAR(64) / VARCHAR(500)	Nullable	Explanation for transition
changed_by_type / changed_by_id	VARCHAR(24) / VARCHAR(128)	Type default SYSTEM	Actor information
created_at	DATETIME(6)	Indexed with order_id	Transition time

`order_idempotency`

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Idempotency record UUID
user_id	CHAR(36)	External reference	Requesting user
idempotency_key	VARCHAR(128)	Unique with user_id	Client retry identity
request_hash	CHAR(64)	Not null	Detects key reuse with a different payload
order_id	CHAR(36)	Nullable FK → orders.id	Resulting order
created_at / expires_at	DATETIME(6)	Not null	Retention window

`inbox_event`

Column	Type	Key / constraint	Purpose
event_id	CHAR(36)	PK	Deduplication identity
source_service / event_type	VARCHAR(64) / VARCHAR(128)	Not null	Event classification
payload_json	JSON	Not null	Incoming event body
status	VARCHAR(24)	Default RECEIVED	Consumer state
received_at / processed_at	DATETIME(6)	Processed nullable	Consumer timestamps
last_error	VARCHAR(1000)	Nullable	Failure details

`outbox_event`

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Event UUID
aggregate_type / aggregate_id	VARCHAR(64) / CHAR(36)	Not null	Source order aggregate
event_type	VARCHAR(128)	Not null	Domain event name
payload_json	JSON	Not null	Outgoing event body
status	VARCHAR(24)	Default NEW	Relay state
created_at / published_at	DATETIME(6)	Published nullable	Relay timestamps
retry_count / last_error	INT / VARCHAR(1000)	Recovery columns	Relay diagnostics

B.7 Payment Processor Schema

The active payment schema is intentionally smaller than the broader reference design. It contains the current Payment entity, Stripe webhook deduplication and the transactional outbox.

payment

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Payment UUID
order_id	CHAR(36)	Unique, external reference	One active payment row per order
user_id	CHAR(36)	External reference	Payment owner
customer_email	VARCHAR(320)	Nullable, added by V2	Email snapshot for events
status	VARCHAR(40)	Not null	Payment state machine value
currency / amount	CHAR(3) / DECIMAL(19,4)	Not null	Authoritative order amount copied from Order
idempotency_key	VARCHAR(80)	Unique with user_id	Client retry identity
provider_payment_intent_id	VARCHAR(120)	Indexed, nullable	Stripe PaymentIntent ID
provider_status / failure_reason	VARCHAR(60) / VARCHAR(255)	Nullable	Provider outcome
version	BIGINT	Default 0	Optimistic locking version
created_at / updated_at	DATETIME(6)	Not null	UTC audit timestamps

webhook_event

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Local webhook UUID
provider_event_id	VARCHAR(120)	Unique	Stripe event deduplication key
event_type	VARCHAR(80)	Not null	Stripe event name
status	VARCHAR(40)	Indexed	Webhook processing state
payment_id	CHAR(36)	Nullable logical reference	Matched payment; no FK in active migration
received_at / processed_at	DATETIME(6)	Processed nullable	Processing timestamps

outbox_event

Column	Type	Key / constraint	Purpose
id	CHAR(36)	PK	Event UUID
aggregate_id / aggregate_type	CHAR(36) / VARCHAR(40)	Indexed aggregate_id	Source payment aggregate
event_type	VARCHAR(80)	Not null	Payment event name
payload	JSON	Not null	Outgoing event body
status	VARCHAR(20)	Indexed with created_at	Relay state
retry_count / last_error	INT / VARCHAR(500)	Recovery columns	Relay diagnostics
created_at / published_at	DATETIME(6)	Published nullable	Relay timestamps

Appendix C — Implemented Endpoint Catalogue

Chapter 5 explains cross-service contracts and the purchase flow. This appendix lists implemented REST endpoints so routes and access expectations can be checked quickly without repeating the interaction narrative.

C.1 Product Catalog and User Management Endpoints

Method	Path	Purpose / access
GET	/api/v1/products	Search active products with bounded pagination and allowed sorting
GET	/api/v1/products/{productId}	Read product details
POST / PATCH / DELETE	/api/v1/products/{productId}	Create, update or soft-delete a product; administrative use
POST / PATCH / DELETE	/api/v1/products/{productId}/images/{imageId}	Maintain image metadata
GET / POST / PATCH / DELETE	/api/v1/categories/{categoryId}	List and maintain categories
PUT	/internal/v1/inventory/{productId}	Adjust on-hand stock
POST	/internal/v1/inventory/reservations	Reserve all requested lines atomically
DELETE	/internal/v1/inventory/reservations/{orderId}	Idempotently release active reservations
POST	/api/v1/auth/signup	Register a customer
POST	/api/v1/auth/login	Return access and refresh tokens
POST	/api/v1/auth/refresh	Rotate a valid refresh session
POST	/api/v1/auth/logout	Revoke the refresh session
GET / PATCH	/api/v1/users/me	Read or update the authenticated profile
GET / POST / PATCH / DELETE	/api/v1/users/me/addresses/{addressId}	Maintain the current user's addresses
GET	/internal/v1/users/{userId}	Small internal user projection

C.2 Order and Payment Endpoints

Method	Path	Purpose / access
POST	/api/v1/orders	Create an order using Idempotency-Key and authenticated user
GET	/api/v1/orders	List orders visible to the current user
GET	/api/v1/orders/{orderId}	Return an owned order with item and address snapshots
GET	/api/v1/orders/{orderId}/status	Return the current lifecycle state
GET	/internal/v1/orders/{orderId}/payment-details	Return authoritative owner, amount, currency and payable state
POST	/internal/v1/outbox/{id}/requeue	Administrative recovery of a failed outbox record
POST	/api/v1/payments	Create an idempotent Stripe test-mode PaymentIntent
GET	/api/v1/payments/{paymentId}	Read a payment owned by the caller or by an administrator
POST	/webhooks/stripe	Accept a signature-verified Stripe event

Validation errors, missing resources, state conflicts and dependency failures use a common `ApiErrorDTO` shape inside each REST service. `CorrelationIdFilter` accepts or creates a request correlation ID and places it on responses and logs, which makes a call traceable across service boundaries.

Appendix D — Additional Verification Cases

Chapter 5 contains the principal requirement-to-test traceability matrix. The short table below records only additional cases that are not listed in that core matrix.

D.1 Additional Test Evidence

Requirement	Representative test evidence	What the test checks
Multi-line reservation is atomic	InventoryReservationTest	One unavailable line rolls back the entire reservation.
Product API behaves consistently	ProductControllerTest; ProductCatalogIntegrationTest	Validation, mapping, persistence and response behaviour.
Active migrations match MySQL	*MysqlMigrationIT tests	Flyway scripts apply against the intended database engine.